

## 17 Fourierova transformace

Co má společného násobení polynomů s kompresí zvuku? Nebo třeba s rozpoznáváním obrazu? V této kapitole ukážeme, že na pozadí všech těchto otázek je společná algebraická struktura, kterou matematici znají pod názvem *diskrétní Fourierova transformace*. Odvodíme efektivní algoritmus pro výpočet této transformace a ukážeme některé jeho zajímavé důsledky.

### 17.1 Polynomy a jejich násobení

Nejprve stručně připomeňme, jak se pracuje s polynomy.

**Definice:** *Polynom* je výraz typu

$$P(x) = \sum_{i=0}^{n-1} p_i \cdot x^i,$$

kde  $x$  je proměnná a  $p_0$  až  $p_{n-1}$  jsou čísla, kterým říkáme *koefficienty* polynomu. Zde budeme značit polynomy velkými písmeny a jejich koeficienty příslušnými malými písmeny s indexy. Zatím budeme předpokládat, že všechna čísla jsou reálná, v obecnosti by to mohly být prvky libovolného komutativního okruhu.

V algoritmech obvykle polynomy reprezentujeme *vektorem koeficientů*  $(p_0, \dots, p_{n-1})$ ; oproti zvyklostem lineární algebry budeme složky vektorů v celé této kapitole indexovat od 0. Počtu koeficientů  $n$  budeme říkat *velikost polynomu*  $|P|$ . Časovou složitost algoritmu budeme vyjadřovat vzhledem k velikostem polynomů zadaných na vstupu. Budeme předpokládat, že s reálnými čísly umíme pracovat v konstantním čase na operaci.

Pokud přidáme nový koeficient  $p_n = 0$ , hodnota polynomu se pro žádné  $x$  nezmění. Stejně tak je-li nejvyšší koeficient  $p_{n-1}$  nulový, můžeme ho vynechat. Takto můžeme každý polynom zmenšit na *normální tvar*, v němž má buďto nenulový nejvyšší koeficient, nebo nemá vůbec žádné koeficienty – to je takzvaný *nulový polynom*, který je pro každé  $x$  roven nule. Nejvyšší mocnině s nenulovým koeficientem se říká *stupeň polynomu*  $\deg P$ , nulovému polynomu přiřazujeme stupeň  $-1$ .

S polynomy zacházíme jako s výrazy. Sčítání a odečítání je přímočaré, ale podívejme se, co se děje při *násobení*:

$$P(x) \cdot Q(x) = \left( \sum_{i=0}^{n-1} p_i \cdot x^i \right) \cdot \left( \sum_{j=0}^{m-1} q_j \cdot x^j \right) = \sum_{i,j} p_i q_j x^{i+j}.$$

Tento součin můžeme zapsat jako polynom  $R(x)$ , jehož koeficient u  $x^k$  je roven  $r_k = p_0q_k + p_1q_{k-1} + \dots + p_kq_0$ . Nahlédneme, že polynom  $R$  má stupeň  $\deg P + \deg Q$  a velikost  $|P| + |Q| - 1$ .

Algoritmus, který počítá součin dvou polynomů velikosti  $n$  přímo podle definice, proto spotřebuje čas  $\Theta(n)$  na výpočet každého koeficientu, takže celkem  $\Theta(n^2)$ . Podobně jako u násobení čísel, i zde se budeme snažit najít efektivnější způsob.

### Grafy polynomů

Odbočme na chvíli a uvažujme, kdy dva polynomy považujeme za stejné. Na to se dá nahlížet více způsoby. Buďto se na polynomy můžeme dívat jako na výrazy a porovnávat jejich symbolické zápisy. Pak jsou si dva polynomy rovny právě tehdy, mají-li po normalizaci stejné vektory koeficientů. Tehdy říkáme, že jsou *identické* a obvykle to značíme  $P \equiv Q$ .

Nebo můžeme porovnávat polynomy jako reálné funkce. Polynomy  $P$  a  $Q$  jsou si rovny ( $P = Q$ ) právě tehdy, je-li  $P(x) = Q(x)$  pro všechna  $x \in \mathbb{R}$ . Identicky rovné polynomy si jsou rovny i jako funkce, ale musí to platit i naopak? Následující věta ukáže, že ano, a že dokonce stačí rovnost ověřit pro konečný počet  $x$ .

**Věta:** Budte  $P$  a  $Q$  polynomy stupně nejvýše  $d$ . Pokud platí  $P(x_i) = Q(x_i)$  pro navzájem různá čísla  $x_0, \dots, x_d$ , pak  $P$  a  $Q$  jsou identické.

*Důkaz:* Připomeňme nejprve následující klasické lemma o kořenech polynomů:

**Lemma:** Polynom  $R$  stupně  $t \geq 0$  má nejvýše  $t$  kořenů (čísel  $\alpha$ , pro něž je  $R(\alpha) = 0$ ).

*Důkaz:* Pokud vydělíme polynom  $R$  polynomem  $x - \alpha$  (viz cvičení 1), dostaneme  $R(x) \equiv (x - \alpha) \cdot R'(x) + \beta$ , kde  $R'$  je polynom stupně  $t - 1$  a  $\beta$  je konstanta. Je-li  $\alpha$  kořenem  $R$ , musí být  $\beta = 0$ , takže  $R(x) \equiv (x - \alpha) \cdot R'(x)$ .

Úplně stejně pokračujeme s  $R'$ , až získáme  $R(x) = (x - \alpha_1) \cdot \dots \cdot (x - \alpha_k) \cdot R''(x)$ , kde  $R''(x)$  nemá žádné kořeny. Všechny kořeny polynomu  $R$  jsou tedy čísla  $\alpha_1, \dots, \alpha_k$ . Pravá strana rovnosti má ovšem stupeň aspoň  $k$ , takže aby se mohla rovnat  $R$ , musí být  $k \leq t$ .  $\square$

Abychom dokázali větu, stačí uvážít polynom  $R(x) \equiv P(x) - Q(x)$ . Tento polynom má stupeň nejvýše  $d$ , ovšem každé z čísel  $x_0, \dots, x_d$  je jeho kořenem. Podle lemmatu musí tedy být identicky nulový, a proto  $P \equiv Q$ .  $\square$

Díky předchozí větě můžeme polynomy reprezentovat nejen vektorem koeficientů, ale také *vektorem funkčních hodnot* v nějakých smluvených bodech – tomuto vektoru budeme

říkat *graf polynomu*. Pokud zvolíme dostatečně mnoho bodů, je polynom svým grafem jednoznačně určen.

V této reprezentaci je násobení polynomů triviální: Součin polynomů  $P$  a  $Q$  má v bodě  $x$  hodnotu  $P(x) \cdot Q(x)$ . Stačí tedy grafy vynásobit po složkách, což zvládneme v lineárním čase. Jen je potřeba dát pozor na to, že součin má vyšší stupeň než jednotliví činitelé, takže musíme polynomy vyhodnocovat ve dvojnásobném počtu bodů.

#### Algoritmus NÁSOBENÍ POLYNOMŮ

1. Jsou dány polynomy  $P$  a  $Q$  velikosti  $n$ , určené svými koeficienty. Bez újmy na obecnosti předpokládejme, že horních  $n/2$  koeficientů je u obou polynomů nulových, takže součin  $R \equiv P \cdot Q$  bude také polynom velikosti  $n$ .
2. Zvolíme navzájem různá čísla  $x_0, \dots, x_{n-1}$ .
3. Spočítáme grafy polynomů  $P$  a  $Q$ , čili vektory  $(P(x_0), \dots, P(x_{n-1}))$  a  $(Q(x_0), \dots, Q(x_{n-1}))$ .
4. Z toho vypočteme graf součinu  $R$  vynásobením po složkách:  $R(x_i) = P(x_i) \cdot Q(x_i)$ .
5. Nalezneme koeficienty polynomu  $R$  tak, aby odpovídaly grafu.

Krok 4 trvá  $\Theta(n)$ , takže rychlost celého algoritmu stojí a padá s efektivitou převodů mezi koeficientovou a hodnotovou reprezentací polynomů. To obecně neumíme v lepším než kvadratickém čase, ale zde máme možnost volby bodů  $x_0, \dots, x_{n-1}$ , takže si je zvolíme tak šikovně, aby převod šel provést rychle.

#### Vyhodnocení polynomu metodou Rozděl a panuj

Nyní se pokusíme sestavit algoritmus pro vyhodnocení polynomu založený na metodě Rozděl a panuj. Sice tento pokus nakonec selže, ale bude poučné podívat se, proč a jak selhal.

Uvažujme polynom  $P$  velikosti  $n$ , který chceme vyhodnotit v  $n$  bodech. Body si zvolíme tak, aby byly *spárované*, tedy aby tvořily dvojice lišící se pouze znaménkem:

$$\pm x_0, \pm x_1, \dots, \pm x_{n/2-1}.$$

Polynom  $P$  můžeme rozložit na členy se sudými exponenty a na ty s lichými:

$$P(x) = (p_0x^0 + p_2x^2 + \dots + p_{n-2}x^{n-2}) + (p_1x^1 + p_3x^3 + \dots + p_{n-1}x^{n-1}).$$

Navíc můžeme z druhé závorky vytknout  $x$ :

$$P(x) = (p_0x^0 + p_2x^2 + \dots + p_{n-2}x^{n-2}) + x \cdot (p_1x^0 + p_3x^2 + \dots + p_{n-1}x^{n-2}).$$

V obou závorkách se nyní vyskytují pouze sudé mocniny  $x$ . Proto můžeme každou závorku považovat za vyhodnocení nějakého polynomu velikosti  $n/2$  v bodě  $x^2$ , tedy:

$$P(x) = P_s(x^2) + x \cdot P_\ell(x^2),$$

kde:

$$\begin{aligned} P_s(t) &= p_0 t^0 + p_2 t^1 + \dots + p_{n-2} t^{\frac{n-2}{2}}, \\ P_\ell(t) &= p_1 t^0 + p_3 t^1 + \dots + p_{n-1} t^{\frac{n-2}{2}}. \end{aligned}$$

Navíc pokud podobným způsobem dosadíme do  $P$  hodnotu  $-x$ , dostaneme:

$$P(-x) = P_s(x^2) - x \cdot P_\ell(x^2).$$

Vyhodnocení polynomu  $P$  v bodech  $\pm x_0, \dots, \pm x_{n/2-1}$  tedy můžeme převést na vyhodnocení polynomů  $P_s$  a  $P_\ell$  poloviční velikosti v bodech  $x_0^2, \dots, x_{n/2-1}^2$ .

To naznačuje algoritmus s časovou složitostí  $T(n) = 2T(n/2) + \Theta(n)$  a z Kuchařkové věty víme, že taková rekurence má řešení  $T(n) = \Theta(n \log n)$ . Jenže ovšem, tento algoritmus nefunguje: druhé mocniny, které předáme rekurzivnímu volání, jsou vždy nezáporné, takže už nemohou být správně spárovány. Tedy ... alespoň dokud počítáme s reálnými čísly.

Ukážeme, že v oboru komplexních čísel už můžeme zvolit body, které budou správně spárovány i po několikerém umocnění na druhou.

### Cvičení

1. Odvoďte dělení polynomů se zbytkem: Jsou-li  $P$  a  $Q$  polynomy a  $\deg Q > 0$ , pak existují polynomy  $R$  a  $S$  takové, že  $P \equiv QR + S$  a  $\deg S < \deg Q$ . Zkuste pro toto dělení nalézt co nejefektivnější algoritmus.
2. Převod grafu na polynom v obecném případě: Hledáme polynom stupně nejvýše  $n$ , který prochází body  $(x_0, y_0), \dots, (x_n, y_n)$  pro  $x_i$  navzájem různá. Pomůže *Lagrangeova interpolace*: definujme polynomy

$$A_j(x) = \prod_{k \neq j} (x - x_k), \quad B_j(x) = \frac{A_j(x)}{\prod_{k \neq j} (x_j - x_k)}, \quad P(x) = \sum_j y_j B_j(x).$$

Dokažte, že  $\deg P \leq n$  a  $P(x_j) = y_j$  pro všechna  $j$ . K tomu pomůže rozmyslet si, jak vyjde  $A_j(x_k)$  a  $B_j(x_k)$ .

3. Sestrojte co nejrychlejší algoritmus pro Lagrangeovu interpolaci z předchozího cvičení.

4. *Jiný pohled na interpolaci polynomů:* Hledáme-li polynom  $P(x) = \sum_{k=0}^n p_k x^k$  procházející body  $(x_0, y_0), \dots, (x_n, y_n)$ , řešíme vlastně soustavu rovnic tvaru  $\sum_k p_k x_j^k = y_j$  pro  $j = 0, \dots, n$ . Rovnice jsou lineární v neznámých  $p_0, \dots, p_n$ , takže hledáme vektor  $\mathbf{p}$  splňující  $\mathbf{V}\mathbf{p} = \mathbf{y}$ , kde  $\mathbf{V}$  je takzvaná *Vandermondova matice* s  $V_{jk} = x_j^k$ . Dokažte, že pro  $x_j$  navzájem různá je matice  $\mathbf{V}$  regulární, takže soustava rovnic má právě jedno řešení. [N41]
- 5.\* *Generálové:* Pro odpálení jaderné bomby je potřeba, aby se na tom shodlo alespoň  $k$  z celkového počtu  $n$  generálů. Vymyslete, jak z odpalovacího kódu odvodit  $n$  klíčů pro generály tak, aby libovolná skupina  $k$  generálů uměla ze svých klíčů kód vypočítat, ale žádná menší skupina nemohla o kód zjistit nic než jeho délku. [N123]

## 17.2 Intermezzo o komplexních číslech

V této kapitole budeme počítat s komplexními čísly. Zopakujme si proto, jak se s nimi zachází.

### Základní operace

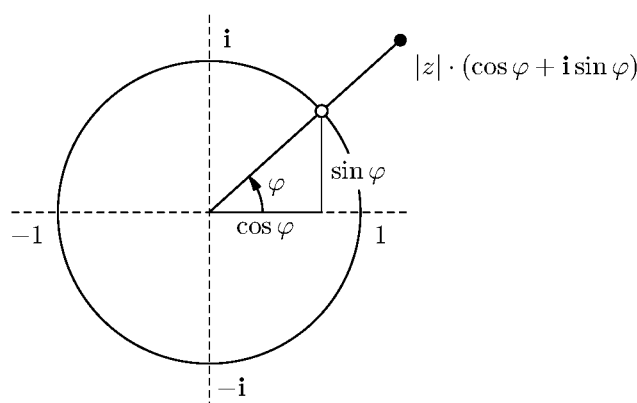
- Definice:  $\mathbb{C} = \{a + b\mathbf{i} \mid a, b \in \mathbb{R}\}$ ,  $\mathbf{i}^2 = -1$ .
- Sčítání a odčítání:  $(a + b\mathbf{i}) \pm (p + q\mathbf{i}) = (a \pm p) + (b \pm q)\mathbf{i}$ .
- Násobení:  $(a + b\mathbf{i})(p + q\mathbf{i}) = ap + aq\mathbf{i} + bp\mathbf{i} + bq\mathbf{i}^2 = (ap - bq) + (aq + bp)\mathbf{i}$ . Pro  $\alpha \in \mathbb{R}$  je  $\alpha(a + b\mathbf{i}) = \alpha a + \alpha b\mathbf{i}$ .
- Komplexní sdružení:  $\overline{a + b\mathbf{i}} = a - b\mathbf{i}$ .  
 $\overline{\overline{x}} = x$ ,  $\overline{x \pm y} = \overline{x} \pm \overline{y}$ ,  $\overline{x \cdot y} = \overline{x} \cdot \overline{y}$ ,  $x \cdot \overline{x} = (a + b\mathbf{i})(a - b\mathbf{i}) = a^2 + b^2 \in \mathbb{R}$ .
- Absolutní hodnota:  $|x| = \sqrt{x \cdot \overline{x}}$ , takže  $|a + b\mathbf{i}| = \sqrt{a^2 + b^2}$ .  
 Také  $|\alpha x| = |\alpha| \cdot |x|$  pro  $\alpha \in \mathbb{R}$  (později uvidíme, že to platí i pro  $\alpha$  komplexní).
- Dělení: Podíl  $x/y$  rozšíříme číslem  $\overline{y}$  na  $(x \cdot \overline{y})/(y \cdot \overline{y})$ . Nyní je jmenovatel reálný, takže můžeme vydělit každou složku čitatele zvlášť.

### Gaussova rovina a goniometrický tvar<sup>(1)</sup>

- Komplexním číslům přiřadíme body v  $\mathbb{R}^2$ :  $a + b\mathbf{i} \leftrightarrow (a, b)$ .
- $|x|$  vyjadřuje vzdálenost od bodu  $(0, 0)$ .

<sup>(1)</sup> Říká se jí podle slavného německého matematika Carla Friedricha Gauße, jenž je známý spíš jako Carolus Fridericus Gauss – svá díla totiž stejně jako většina vědců 18. století psal latinsky.

- $|x| = 1$  pro čísla ležící na jednotkové kružnici (*komplexní jednotky*).  
Pak platí  $x = \cos \varphi + \mathbf{i} \sin \varphi$  pro nějaké  $\varphi \in [0, 2\pi)$ .
- Pro libovolné  $x \in \mathbb{C}$ :  $x = |x| \cdot (\cos \varphi(x) + \mathbf{i} \sin \varphi(x))$ .  
Číslo  $\varphi(x)$  říkáme *argument* čísla  $x$  a někdy ho také značíme  $\arg x$ . Argumenty jsou periodické s periodou  $2\pi$ , často se normalizují do intervalu  $[0, 2\pi)$ .
- Navíc  $\varphi(\bar{x}) = -\varphi(x)$ , uvažujeme-li argumenty modulo  $2\pi$ .



Obrázek 17.1: Goniometrický tvar komplexního čísla

### Exponenciální tvar

- Eulerova formule:  $e^{\mathbf{i}\varphi} = \cos \varphi + \mathbf{i} \sin \varphi$ .
- Každé  $x \in \mathbb{C}$  lze tedy zapsat jako  $|x| \cdot e^{\mathbf{i}\varphi(x)}$ .
- Násobení:  $xy = (|x| \cdot e^{\mathbf{i}\varphi(x)}) \cdot (|y| \cdot e^{\mathbf{i}\varphi(y)}) = |x| \cdot |y| \cdot e^{\mathbf{i}(\varphi(x)+\varphi(y))}$   
(absolutní hodnoty se násobí, argumenty sčítají).
- Umocňování: Pro  $\alpha \in \mathbb{R}$  je  $x^\alpha = (|x| \cdot e^{\mathbf{i}\varphi(x)})^\alpha = |x|^\alpha \cdot e^{\mathbf{i}\alpha\varphi(x)}$ .

### Odmocniny z jedničky

Odmocňování v komplexních číslech není obecně jednoznačné: jestliže třeba budeme hledat čtvrtou odmocninu z jedničky, totiž řešit rovnici  $x^4 = 1$ , nalezneme hned čtyři řešení: 1,  $-1$ ,  $\mathbf{i}$  a  $-\mathbf{i}$ .

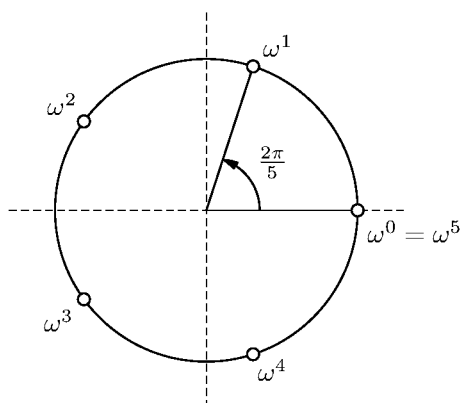
Prozkoumejme nyní obecněji, jak se chovají  $n$ -té odmocniny z jedničky, tedy komplexní kořeny rovnice  $x^n = 1$ :

- Jelikož  $|x^n| = |x|^n$ , musí být  $|x| = 1$ . Proto  $x = e^{i\varphi}$  pro nějaké  $\varphi$ .
- Má platit  $1 = x^n = e^{i\varphi n} = \cos \varphi n + i \sin \varphi n$ . To nastane, kdykoliv  $\varphi n = 2k\pi$  pro nějaké  $k \in \mathbb{Z}$ .

Dostáváme tedy  $n$  různých  $n$ -tých odmocnin z 1, totiž  $e^{2k\pi i/n}$  pro  $k = 0, \dots, n-1$ . Některé z těchto odmocnin jsou ovšem speciální:

**Definice:** Komplexní číslo  $x$  je *primitivní  $n$ -tá odmocnina z 1*, pokud  $x^n = 1$  a žádné z čísel  $x^1, x^2, \dots, x^{n-1}$  není rovno 1.

**Příklad:** Ze čtyř zmíněných čtvrtých odmocnin z 1 jsou  $i$  a  $-i$  primitivní a druhé dvě nikoliv (ověřte sami dosazením). Pro obecné  $n > 2$  vždy existují alespoň dvě primitivní odmocniny, totiž čísla  $\omega = e^{2\pi i/n}$  a  $\bar{\omega} = e^{-2\pi i/n}$ . Platí totiž, že  $\omega^j = e^{2\pi i j/n}$ , a to je rovno 1 právě tehdy, je-li  $j$  násobkem  $n$  (jednotlivé mocniny čísla  $\omega$  postupně obíhají jednotkovou kružnici). Analogicky pro  $\bar{\omega}$ .



Obrázek 17.2: Primitivní pátá odmocnina z jedničky  $\omega$  a její mocniny

**Pozorování:** Pro libovolné číslo  $\omega$ , které je primitivní  $n$ -tou odmocninou z jedničky, platí:

- $\omega^j \neq \omega^k$ , kdykoliv  $0 \leq j < k < n$ . Stačí se podívat na podíl  $\omega^k/\omega^j = \omega^{k-j}$ . Ten nemůže být roven jedné, protože  $0 < k-j < n$  a  $\omega$  je primitivní.
- Pro sudé  $n$  je navíc  $\omega^{n/2} = -1$ . Platí totiž  $(\omega^{n/2})^2 = \omega^n = 1$ , takže  $\omega^{n/2}$  je druhá odmocnina z 1. Takové odmocniny jsou jenom dvě: 1 a  $-1$ , ovšem 1 to být nemůže, protože  $\omega$  je primitivní.

### Cvičení

1. Dokažte, že pro každou primitivní  $n$ -tou odmocninu z jedničky  $\alpha$  platí, že  $\alpha = \omega^t$ , kde  $\omega = e^{2\pi i/n}$  a  $t$  je přirozené číslo nesoudělné s  $n$ . Kolik primitivních  $n$ -tých odmocnin tedy existuje?

## 17.3 Rychlá Fourierova transformace

Ukážeme, že primitivních odmocnin lze využít k záchraně párovacího algoritmu na vyhodnocování polynomů z oddílu 17.1.

Nejprve polynomy doplníme nulami tak, aby jejich velikost  $n$  byla mocninou dvojky. Poté zvolíme nějakou primitivní  $n$ -tou odmocninu z jedničky  $\omega$  a budeme polynom vyhodnocovat v bodech  $\omega^0, \omega^1, \dots, \omega^{n-1}$ . To jsou navzájem různá komplexní čísla, která jsou správně spárovaná: hodnoty  $\omega^{n/2}, \dots, \omega^{n-1}$  se od  $\omega^0, \dots, \omega^{n/2-1}$  liší pouze znaménkem. To snadno ověříme: pro  $0 \leq j < n/2$  je  $\omega^{n/2+j} = \omega^{n/2}\omega^j = -\omega^j$ . Navíc  $\omega^2$  je primitivní  $(n/2)$ -tá odmocnina z jedničky, takže se rekurzivně voláme na problém téhož druhu, který je správně spárovaný.

Náš plán použít metodu Rozděl a panuj tedy nakonec vyšel: opravdu máme algoritmus o složitosti  $\Theta(n \log n)$  pro vyhodnocení polynomu. Ještě ho upravíme tak, aby místo s polynomy pracoval s vektory jejich koeficientů či hodnot. Tomuto algoritmu se říká FFT, vzápětí prozradíme, proč.

### Algoritmus FFT (rychlá Fourierova transformace)

*Vstup:* Číslo  $n = 2^k$ , primitivní  $n$ -tá odmocnina z jedničky  $\omega$ ,  
vektor  $(p_0, \dots, p_{n-1})$  koeficientů polynomu  $P$

1. Pokud  $n = 1$ , položíme  $y_0 \leftarrow p_0$  a skončíme.
2. Jinak se rekurzivně zavoláme na sudou a lichou část koeficientů:
3.  $(s_0, \dots, s_{n/2-1}) \leftarrow \text{FFT}(n/2, \omega^2, (p_0, p_2, p_4, \dots, p_{n-2}))$ .
4.  $(\ell_0, \dots, \ell_{n/2-1}) \leftarrow \text{FFT}(n/2, \omega^2, (p_1, p_3, p_5, \dots, p_{n-1}))$ .
5. Z grafů obou částí poskládáme graf celého polynomu:
6. Pro  $j = 0, \dots, n/2 - 1$ :
7.  $y_j \leftarrow s_j + \omega^j \cdot \ell_j$   $\triangleleft$  mocninu  $\omega^j$  průběžně přepočítáváme
8.  $y_{j+n/2} \leftarrow s_j - \omega^j \cdot \ell_j$

*Výstup:* Graf polynomu  $P$ , tedy vektor  $(y_0, \dots, y_{n-1})$ , kde  $y_j = P(\omega^j)$

Vyhodnotit polynom v mocninách čísla  $\omega$  umíme, ale ještě nejsme v cíli. Potřebujeme umět provést dostatečně rychle i opačný převod – z hodnot na koeficienty. K tomu nám pomůže podívat se na vyhodnocování polynomu trochu abstraktněji jako na zobrazení,

které jednomu vektoru komplexních čísel přiřadí jiný vektor. Toto zobrazení matematici potkávají v mnoha různých kontextech už po staletí a nazývají ho Fourierovou transformací.

**Definice:** *Diskrétní Fourierova transformace (DFT)* je zobrazení  $\mathcal{F} : \mathbb{C}^n \rightarrow \mathbb{C}^n$ , které vektoru  $\mathbf{x}$  přiřadí vektor  $\mathbf{y}$  daný předpisem

$$y_j = \sum_{k=0}^{n-1} x_k \cdot \omega^{jk},$$

kde  $\omega$  je nějaká pevně zvolená primitivní  $n$ -tá odmocnina z jedné. Vektor  $\mathbf{y}$  se nazývá *Fourierův obraz* vektoru  $\mathbf{x}$ .

Jak to souvisí s naším algoritmem? Pokud označíme  $\mathbf{p}$  vektor koeficientů polynomu  $P$ , pak jeho Fourierova transformace  $\mathcal{F}(\mathbf{p})$  není nic jiného než graf tohoto polynomu v bodech  $\omega^0, \dots, \omega^{n-1}$ . To ověříme snadno dosazením do definice. Algoritmus tedy počítá diskrétní Fourierovu transformaci v čase  $\Theta(n \log n)$ . Proto se mu říká FFT – Fast Fourier Transform.

Také si všimněme, že DFT je lineární zobrazení. Jde proto zapsat jako násobení nějakou maticí  $\mathbf{\Omega}$ , kde  $\Omega_{jk} = \omega^{jk}$ . Pro převod grafu na koeficienty potřebujeme najít inverzní zobrazení určené inverzní maticí  $\mathbf{\Omega}^{-1}$ .

Jelikož  $\omega^{-1} = \bar{\omega}$ , pojďme zkusit, zda hledanou inverzní maticí není  $\bar{\mathbf{\Omega}}$ .

**Lemma:**  $\mathbf{\Omega} \cdot \bar{\mathbf{\Omega}} = n \cdot \mathbf{E}$ , kde  $\mathbf{E}$  je jednotková matice.

*Důkaz:* Dosazením do definice a elementárními úpravami:

$$\begin{aligned} (\mathbf{\Omega} \cdot \bar{\mathbf{\Omega}})_{jk} &= \sum_{\ell=0}^{n-1} \Omega_{j\ell} \cdot \bar{\Omega}_{\ell k} = \sum_{\ell=0}^{n-1} \omega^{j\ell} \cdot \overline{\omega^{\ell k}} = \sum_{\ell=0}^{n-1} \omega^{j\ell} \cdot \omega^{-\ell k} \\ &= \sum_{\ell=0}^{n-1} \omega^{j\ell} \cdot (\omega^{-1})^{\ell k} = \sum_{\ell=0}^{n-1} \omega^{j\ell} \cdot \omega^{-\ell k} = \sum_{\ell=0}^{n-1} \omega^{(j-k)\ell}. \end{aligned}$$

Poslední suma je ovšem geometrická řada. Pokud  $j = k$ , jsou všechny členy řady jedničky, takže se sečtou na  $n$ . Pro  $j \neq k$  použijeme známý vztah pro součet geometrické řady s kvocientem  $q = \omega^{j-k}$ :

$$\sum_{\ell=0}^{n-1} q^\ell = \frac{q^n - 1}{q - 1} = \frac{\omega^{(j-k)n} - 1}{\omega^{j-k} - 1} = 0.$$

Poslední rovnost platí díky tomu, že  $\omega^{(j-k)n} = (\omega^n)^{j-k} = 1^{j-k} = 1$ , takže čítec zlomku je nulový; naopak jmenovatel určité nulový není, jelikož  $\omega$  je primitivní a  $0 < |j-k| < n$ .  $\square$

**Důsledek:**  $\Omega^{-1} = (1/n) \cdot \overline{\Omega}$ .

Matice  $\Omega$  tedy je regulární a její inverze se kromě vydělení  $n$  liší pouze komplexním sdružením. Navíc číslo  $\overline{\omega} = \omega^{-1}$  je také primitivní  $n$ -tou odmocninou z jedničky, takže až na faktor  $1/n$  se jedná opět o Fourierovu transformaci a můžeme ji spočítat stejným algoritmem FFT. Shrňme, co jsme zjistili, do následujících vět:

**Věta:** Je-li  $n$  mocnina dvojky, lze v čase  $\Theta(n \log n)$  spočítat diskretní Fourierovu transformaci v  $\mathbb{C}^n$  i její inverzi.

**Věta:** Polynomy velikosti  $n$  nad tělesem  $\mathbb{C}$  lze násobit v čase  $\Theta(n \log n)$ .

*Důkaz:* Nejprve vektory koeficientů doplníme nulami, tak aby jejich délka byla současně mocnina dvojky a alespoň  $2n$ . Pak pomocí DFT v čase  $\Theta(n \log n)$  převedeme oba polynomy na grafy, v  $\Theta(n)$  vynásobíme grafy po složkách a výsledný graf pomocí inverzní DFT v čase  $\Theta(n \log n)$  převedeme zpět na koeficienty polynomu.  $\square$

Fourierova transformace se kromě násobení polynomů hodí i na ledacos jiného. Své uplatnění nachází nejen v dalších algebraických algoritmech, ale také ve fyzikálních aplikacích – odpovídá totiž spektrálnímu rozkladu signálu na siny a cosiny o různých frekvencích. Na tom jsou založeny například algoritmy pro filtrování zvuku, pro kompresi zvuku a obrazu (MP3, JPEG), nebo třeba rozpoznávání řeči. Něco z toho naznačíme ve zbytku této kapitoly.

### Cvičení

1. O jakých vlastnostech vektoru vypovídá nultý a  $(n/2)$ -tý koeficient jeho Fourierova obrazu?
2. Spočítejte Fourierovy obrazy následujících vektorů z  $\mathbb{C}^n$ :
  - $(x, \dots, x)$
  - $(1, -1, 1, -1, \dots, 1, -1)$
  - $(1, 0, 1, 0, 1, 0, 1, 0)$
  - $(\omega^0, \omega^1, \omega^2, \dots, \omega^{n-1})$
  - $(\omega^0, \omega^2, \omega^4, \dots, \omega^{2n-2})$
3. Inspirujte se předchozím cvičením a najděte pro každé  $j$  vektor, jehož Fourierův obraz má na  $j$ -tém místě jedničku a všude jinde nuly. Jak z toho přímo sestojit inverzní transformaci?
- 4.\* Co vypovídá o vektoru  $(n/4)$ -tý koeficient jeho Fourierova obrazu?

5. Mějme vektor  $\mathbf{y}$ , který vznikl rotací vektoru  $\mathbf{x}$  o  $k$  pozic ( $y_j = x_{(j+k) \bmod n}$ ). Jak spolu souvisí  $\mathcal{F}(\mathbf{x})$  a  $\mathcal{F}(\mathbf{y})$ ?
6. *Fourierova báze:* Uvažujme systém vektorů  $\mathbf{b}^0, \dots, \mathbf{b}^{n-1}$  se složkami  $\mathbf{b}_k^j = \bar{\omega}^{jk}/\sqrt{n}$ . Dokažte, že tyto vektory tvoří ortonormální bázi prostoru  $\mathbb{C}^n$ , pokud použijeme standardní skalární součin nad  $\mathbb{C}$ :  $\langle \mathbf{x}, \mathbf{y} \rangle = \sum_j x_j \bar{y}_j$ . Složky vektoru  $\mathbf{x}$  vzhledem k této bázi pak jsou  $\langle \mathbf{x}, \mathbf{b}^0 \rangle, \dots, \langle \mathbf{x}, \mathbf{b}^{n-1} \rangle$  (to platí pro libovolnou ortonormální bázi). Uvědomte si, že tyto skalární součiny odpovídají definici DFT, tedy až na konstantu  $1/\sqrt{n}$ .
- 7.\* *Volba  $\omega$ :* Ve Fourierově transformaci máme volnost v tom, jakou primitivní odmocninu  $\omega$  si vybereme. Ukažte, že Fourierovy obrazy pro různé volby  $\omega$  se liší pouze pořadím složek. [N72]

## 17.4\* Spektrální rozklad

Ukážeme, jak FFT souvisí s digitálním zpracováním signálu – pro jednoduchost jedno-rozměrného, tedy třeba zvuku.

Uvažujme reálnou funkci  $f$  definovanou na intervalu  $[0, 1)$ . Pokud její hodnoty *navzorkujeme* v  $n$  pravidelně rozmístěných bodech, získáme vektor  $\mathbf{f} \in \mathbb{R}^n$  o složkách  $\mathbf{f}_j = f(j/n)$ . Co o funkci  $f$  vypovídá Fourierův obraz vektoru  $\mathbf{f}$ ?

**Lemma R (DFT reálného vektoru):** Je-li  $\mathbf{x}$  reálný vektor z  $\mathbb{R}^n$ , jeho Fourierův obraz  $\mathbf{y} = \mathcal{F}(\mathbf{x})$  je *antisymetrický*:  $\mathbf{y}_j = \bar{\mathbf{y}}_{n-j}$  pro všechna  $j$ . (Připomínáme, že vektory v této kapitole indexujeme modulo  $n$ , takže  $\mathbf{y}_n = \mathbf{y}_0$ .)

*Důkaz:* Z definice DFT víme, že

$$\mathbf{y}_{n-j} = \sum_k \mathbf{x}_k \omega^{(n-j)k} = \sum_k \mathbf{x}_k \omega^{nk-jk} = \sum_k \mathbf{x}_k \omega^{-jk} = \sum_k \mathbf{x}_k \bar{\omega}^{jk}.$$

Jelikož komplexní sdružení lze distribuovat přes aritmetické operace, můžeme psát  $\bar{\mathbf{y}}_{n-j} = \sum_k \bar{\mathbf{x}}_k \cdot \omega^{jk}$ , což je pro reálné  $\mathbf{x}$  rovno  $\sum_k \mathbf{x}_k \omega^{jk} = \mathbf{y}_j$ .  $\square$

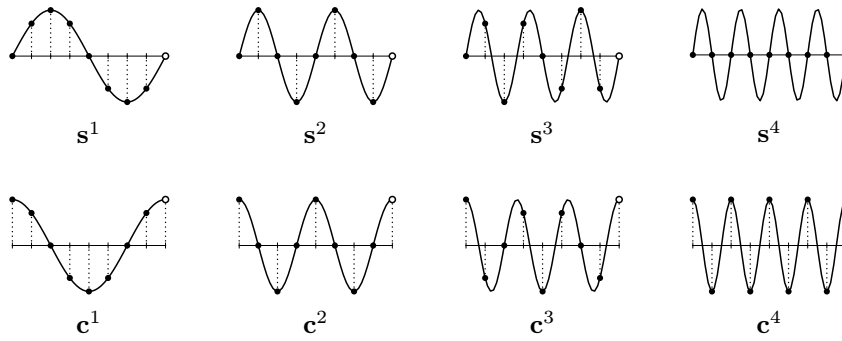
**Důsledek:** Speciálně  $\mathbf{y}_0 = \bar{\mathbf{y}}_0$  a  $\mathbf{y}_{n/2} = \bar{\mathbf{y}}_{n/2}$ , takže obě tyto hodnoty jsou reálné. (To je vidět i z cvičení 17.3.1.)

**Lemma A (o antisymetrických vektorech):** Antisymetrické vektory v  $\mathbb{C}^n$  tvoří vektorový prostor dimenze  $n$  nad tělesem reálných čísel.

*Důkaz:* Ověříme axiomy vektorového prostoru. (To, že prostor budujeme nad  $\mathbb{R}$ , a nikoliv nad  $\mathbb{C}$ , je důležité: násobení vektoru komplexním skalárem obecně nezachovává antisymetrii.)

Co se dimenze týče: V antisymetrickém vektoru  $\mathbf{y}$  jsou složky  $y_0$  a  $y_{n/2}$  reálné, u složek  $y_1, \dots, y_{n/2-1}$  můžeme volit jak reálnou, tak imaginární část. Ostatní složky tím jsou už jednoznačně dány. Vektor je tedy určen  $n$  nezávislými reálnými parametry.  $\square$

**Definice:** V dalším textu zvolme pevné  $n$  a  $\omega = e^{2\pi i/n}$ . Označíme  $\mathbf{e}^k$ ,  $\mathbf{s}^k$  a  $\mathbf{c}^k$  vektory získané navzorkováním funkcí  $e^{2k\pi i x}$ ,  $\sin 2k\pi x$  a  $\cos 2k\pi x$  (komplexní exponenciála, sinus a cosinus s frekvencí  $k$ ) v  $n$  bodech intervalu  $[0, 1)$ .



Obrázek 17.3: Vektory  $\mathbf{s}^k$  a  $\mathbf{c}^k$  při vzorkování v 8 bodech

**Lemma V (o vzorkování funkcí):** Fourierův obraz vektorů  $\mathbf{e}^k$ ,  $\mathbf{s}^k$  a  $\mathbf{c}^k$  vypadá pro  $0 < k < n/2$  následovně:

$$\begin{aligned}\mathcal{F}(\mathbf{e}^k) &= (0, \dots, 0, n, 0, \dots, 0), \\ \mathcal{F}(\mathbf{s}^k) &= (0, \dots, 0, n/2i, 0, \dots, 0, -n/2i, 0, \dots, 0), \\ \mathcal{F}(\mathbf{c}^k) &= (0, \dots, 0, n/2, 0, \dots, 0, n/2, 0, \dots, 0),\end{aligned}$$

přičemž první vektor má nenulu na pozici  $n - k$ , další dva na pozicích  $k$  a  $n - k$ .

Zatímco vztah pro  $\mathcal{F}(\mathbf{e}^k)$  funguje i s  $k = 0$  a  $k = n/2$ , siny a cosiny se chovají odlišně:  $\mathbf{s}^0$  i  $\mathbf{s}^{n/2}$  jsou nulové vektory, takže  $\mathcal{F}(\mathbf{s}^0)$  a  $\mathcal{F}(\mathbf{s}^{n/2})$  jsou také nulové;  $\mathbf{c}^0$  je vektor samých jedniček s  $\mathcal{F}(\mathbf{c}^0) = (n, 0, \dots, 0)$  a  $\mathbf{c}^{n/2} = (1, -1, \dots, 1, -1)$  s  $\mathcal{F}(\mathbf{c}^{n/2}) = (0, \dots, 0, n, 0, \dots, 0)$  s  $n$  na pozici  $n/2$ .

*Důkaz:* Pro  $\mathbf{e}^k$  si stačí všimnout, že  $\mathbf{e}_j^k = e^{2k\pi i \cdot j/n} = e^{jk \cdot 2\pi i/n} = \omega^{jk}$ . Proto  $t$ -tá složka Fourierova obrazu vyjde  $\sum_j \omega^{jk} \omega^{jt} = \sum_j \omega^{j(k+t)}$ . To je opět geometrická řada, podobně jako u odvození inverzní FT. Pro  $t = n - k$  se sečte na  $n$ , všude jinde na 0. Vektory  $\mathbf{s}^k$  a  $\mathbf{c}^k$  necháváme jako cvičení.  $\square$

Všimněme si nyní, že reálnou lineární kombinací vektorů  $\mathcal{F}(\mathbf{s}^1), \dots, \mathcal{F}(\mathbf{s}^{n/2-1})$  a  $\mathcal{F}(\mathbf{c}^0), \dots, \mathcal{F}(\mathbf{c}^{n/2})$  můžeme získat libovolný antisymetrický vektor. Jelikož DFT je lineární, plyne z toho, že lineární kombinací  $\mathbf{s}^1, \dots, \mathbf{s}^{n/2-1}$  a  $\mathbf{c}^0, \dots, \mathbf{c}^{n/2}$  lze získat libovolný reálný vektor. Přesněji to říká následující věta:

**Věta:** Pro každý vektor  $\mathbf{x} \in \mathbb{R}^n$  existují reálné koeficienty  $\alpha_0, \dots, \alpha_{n/2}$  a  $\beta_0, \dots, \beta_{n/2}$  takové, že:

$$\mathbf{x} = \sum_{k=0}^{n/2} (\alpha_k \mathbf{c}^k + \beta_k \mathbf{s}^k). \quad (*)$$

Tyto koeficienty jdou navíc vypočítat z Fourierova obrazu

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) = (a_0 + b_0 \mathbf{i}, \dots, a_{n-1} + b_{n-1} \mathbf{i})$$

takto:

$$\begin{aligned} \alpha_0 &= a_0/n, \\ \alpha_j &= 2a_j/n \quad \text{pro } j = 1, \dots, n/2 - 1, \\ \alpha_{n/2} &= a_{n/2}/n, \\ \beta_0 &= \beta_{n/2} = 0, \\ \beta_j &= -2b_j/n \quad \text{pro } j = 1, \dots, n/2 - 1. \end{aligned}$$

*Důkaz:* Jelikož DFT má inverzi, můžeme bez obav fourierovat obě strany rovnice (\*). Tedy chceme, aby platilo  $\mathbf{y} = \mathcal{F}(\sum_k \alpha_k \mathbf{s}^k + \beta_k \mathbf{c}^k)$ . Suma na pravé straně je přitom díky linearitě  $\mathcal{F}$  rovna  $\sum_k (\alpha_k \mathcal{F}(\mathbf{s}^k) + \beta_k \mathcal{F}(\mathbf{c}^k))$ . Označme tento vektor  $\mathbf{z}$  a za vydatné pomoci lemmatu **V** vypočítejme jeho složky:

- K  $\mathbf{z}_0$  přispívá pouze  $\mathbf{c}^0$  (ostatní  $\mathbf{s}^k$  a  $\mathbf{c}^k$  mají nultou složku nulovou). Takže  $\mathbf{z}_0 = \alpha_0 \mathbf{c}_0^0 = (a_0/n) \cdot n = a_0$ .
- K  $\mathbf{z}_j$  pro  $j = 1, \dots, n/2 - 1$  přispívají pouze  $\mathbf{c}^j$  a  $\mathbf{s}^j$ :  $\mathbf{z}_j = \alpha_j \mathbf{c}_j^j + \beta_j \mathbf{s}_j^j = 2a_j/n \cdot n/2 - 2b_j/n \cdot n/2 \mathbf{i} = a_j + b_j \mathbf{i}$ .
- K  $\mathbf{z}_{n/2}$  přispívá pouze  $\mathbf{c}^{n/2}$ , takže analogicky vyjde  $\mathbf{z}_{n/2} = a_{n/2}/n \cdot n = a_{n/2}$ .

Vektory  $\mathbf{z}$  a  $\mathbf{y}$  se tedy shodují v prvních  $n/2 + 1$  složkách (nezapomeňte, že  $b_0 = b_{n/2} = 0$ ). Jelikož jsou oba antisymetrické, musí se shodovat i ve zbývajících složkách.  $\square$

**Důsledek:** Pro libovolnou reálnou funkci  $f$  na intervalu  $[0, 1)$  existuje lineární kombinace funkcí  $\sin 2k\pi x$  a  $\cos 2k\pi x$  pro  $k = 0, \dots, n/2$ , která není při vzorkování v  $n$  bodech od dané funkce  $f$  rozlišitelná.

To je diskretní ekvivalent známého tvrzení o spojitě Fourierově transformaci, podle nějž každou „dostatečně hladkou“ periodickou funkci lze lineárně nakombinovat ze sinů a cosinů o celočíselných frekvencích.

To se hodí například při zpracování zvuku: jelikož  $\alpha \cos x + \beta \sin x = A \sin(x + \varphi)$  pro vhodné  $A$  a  $\varphi$ , můžeme kterýkoliv zvuk rozložit na sinusové tóny o různých frekvencích. U každého tónu získáme jeho amplitudu  $A$  a fázový posun  $\varphi$ , což je vlastně (až na nějaký násobek  $n$ ) absolutní hodnota a argument původního komplexního Fourierova koeficientu. Tomu se říká *spektrální rozklad* signálu a díky FFT ho můžeme z navzorkovaného signálu spočítat velmi rychle.

### Cvičení

1. Dokažte „inverzní“ lemma **R**: DFT antisymetrického vektoru je vždy reálná.
2. Dokažte zbytek lemmatu **V**: Jak vypadá  $\mathcal{F}(\mathbf{s}^k)$  a  $\mathcal{F}(\mathbf{c}^k)$ ? [N17]
- 3.\* Analogií DFT pro reálné vektory je *diskretní cosinová transformace (DCT)*. Z DFT v  $\mathbb{C}^n$  odvodíme DCT v  $\mathbb{R}^{n/2+1}$ . Vektor  $(x_0, \dots, x_{n/2})$  doplníme jeho zrcadlovou kopií na  $\mathbf{x} = (x_0, x_1, \dots, x_{n/2}, x_{n/2-1}, \dots, x_1)$ . To je reálný a antisymetrický vektor, takže jeho Fourierův obraz  $\mathbf{y} = \mathcal{F}(\mathbf{x})$  musí být podle lemmatu **R** a cvičení 1 také reálný a antisymetrický:  $\mathbf{y} = (y_0, y_1, \dots, y_{n/2}, y_{n/2-1}, \dots, y_1)$ . Vektor  $(y_0, \dots, y_{n/2})$  prohlásíme za výsledek DCT.  
  
Rozepsáním  $\mathcal{F}^{-1}(\mathbf{y})$  podle definice dokažte, že tento výsledek popisuje, jak zapsat vektor  $\mathbf{x}$  jako lineární kombinaci cosinových vektorů  $\mathbf{c}^0, \dots, \mathbf{c}^{n/2}$ . Oproti DFT tedy používáme pouze cosiny, zato však o dvojnásobném rozsahu frekvencí. [N7]
4. *Konvoluce* vektorů  $\mathbf{x}$  a  $\mathbf{y}$  je vektor  $\mathbf{z} = \mathbf{x} * \mathbf{y}$  takový, že  $z_j = \sum_k \mathbf{x}_k \mathbf{y}_{j-k}$ , přičemž indexujeme modulo  $n$ . Tuto sumu si můžeme představit jako skalární součin vektoru  $\mathbf{x}$  s vektorem  $\mathbf{y}$  napsaným pozpátku a zrotovaným o  $j$  pozic. Konvoluce nám tedy řekne, jak tyto „přetočené skalární součiny“ vypadají pro všechna  $j$ . Dokažte následující vlastnosti: [N122]
  - a)  $\mathbf{x} * \mathbf{y} = \mathbf{y} * \mathbf{x}$  (*komutativita*)
  - b)  $\mathbf{x} * (\mathbf{y} * \mathbf{z}) = (\mathbf{x} * \mathbf{y}) * \mathbf{z}$  (*asociativita*)
  - c)  $\mathbf{x} * (\alpha \mathbf{y} + \beta \mathbf{z}) = \alpha(\mathbf{x} * \mathbf{y}) + \beta(\mathbf{x} * \mathbf{z})$  (*bilinearita*)
  - d)  $\mathcal{F}(\mathbf{x} * \mathbf{y}) = \mathcal{F}(\mathbf{x}) \odot \mathcal{F}(\mathbf{y})$ , kde  $\odot$  je součin vektorů po složkách. To nám dává algoritmus pro výpočet konvoluce v čase  $\Theta(n \log n)$ .
5. *Vyhlazování signálu*: Mějme vektor  $\mathbf{x}$  naměřených dat. Obvyklý způsob, jak je vyčistit od šumu, je transformace typu  $\mathbf{y}_j = \frac{1}{4}\mathbf{x}_{j-1} + \frac{1}{2}\mathbf{x}_j + \frac{1}{4}\mathbf{x}_{j+1}$ . Ta „obrousí špičky“ tím, že každou hodnotu zprůměruje s okolními. Pokud budeme  $\mathbf{x}$  indexovat cyklicky, jedná se o konvoluci  $\mathbf{x} * \mathbf{z}$ , kde  $\mathbf{z}$  je maska tvaru  $(\frac{1}{2}, \frac{1}{4}, 0, \dots, 0, \frac{1}{4})$ .

Fourierův obraz  $\mathcal{F}(\mathbf{z})$  nám říká, jak vyhlazování ovlivňuje spektrum. Například pro  $n = 8$  vyjde  $\mathcal{F}(\mathbf{z}) \approx (1, 0.854, 0.5, 0.146, 0, 0.146, 0.5, 0.854)$ , takže stejnosměrná složka signálu  $\mathbf{c}^0$  zůstane nezměněna, naopak nejvyšší frekvence  $\mathbf{c}^4$  zcela zmizí a pro ostatní  $\mathbf{c}^k$  a  $\mathbf{s}^k$  platí, že čím vyšší frekvence, tím víc je tlumena. Tomu se říká *dolní propust* – nízké frekvence propustí, vysoké omezuje. Jak vypadá propust, která vynuluje  $\mathbf{c}^4$  a ostatní frekvence propustí beze změny?

6. Zpět k polynomům: Uvědomte si, že to, co se děje při násobení polynomů s jejich koeficienty, je také konvoluce. Jen musíme doplnit vektory nulami, aby se neprojevala cykličnost indexování. Takže vztah  $\mathcal{F}(\mathbf{x} * \mathbf{y}) = \mathcal{F}(\mathbf{x}) \odot \mathcal{F}(\mathbf{y})$  z cvičení 4 je jenom jiný zápis našeho algoritmu na rychlé násobení polynomů.

- 7.\* *Diagonalizace*: Mějme vektorový prostor dimenze  $n$  a lineární zobrazení  $f$  v něm. Zvolíme-li nějakou bázi, můžeme vektory zapisovat jako  $n$ -tice čísel a zobrazení  $f$  popsat jako násobení  $n$ -tice vhodnou maticí  $\mathbf{A}$  tvaru  $n \times n$ . Někdy se povede najít bázi z vlastních vektorů, vzhledem k níž je matice  $\mathbf{A}$  *diagonální* – má nenulová čísla pouze na diagonále. Tehdy umíme součin  $\mathbf{A}\mathbf{x}$  spočítat v čase  $\Theta(n)$ . Pro jeden součin se to málokdy vyplatí, protože převod mezi bázemi bývá pomalý, ale pokud jich chceme počítat hodně, pomůže to.

Rozmyslete si, že podobně se můžeme dívat na DFT. Konvoluce je bilineární funkce na  $\mathbb{C}^n$ , což znamená, že je lineární v každém parametru zvlášť. Zvolíme-li bázi, můžeme každou bilineární funkci popsat trojrozměrnou tabulkou  $n \times n \times n$  čísel (to už není matice, ale *tenzor třetího řádu*). Vztah  $\mathcal{F}(\mathbf{x} * \mathbf{y}) = \mathcal{F}(\mathbf{x}) \odot \mathcal{F}(\mathbf{y})$  pak můžeme vyložit takto:  $\mathcal{F}$  převádí vektory z kanonické báze do *Fourierovy báze* (viz cvičení 17.3.6), vzhledem k níž je tenzor konvoluce diagonální (má jedničky na „tělesové úhlopříčce“ a všude jinde nuly). Pomocí FFT pak můžeme mezi bázemi převádět v čase  $\Theta(n \log n)$ .

- 8.\* Při zpracování obrazu se hodí *dvojměrná DFT*, která matici  $\mathbf{X} \in \mathbb{C}^{n \times n}$  přiřadí matici  $\mathbf{Y} \in \mathbb{C}^{n \times n}$  takto ( $\omega$  je opět primitivní  $n$ -tá odmocnina z jedné):

$$\mathbf{Y}_{jk} = \sum_{u,v} \mathbf{X}_{uv} \omega^{ju+kv}.$$

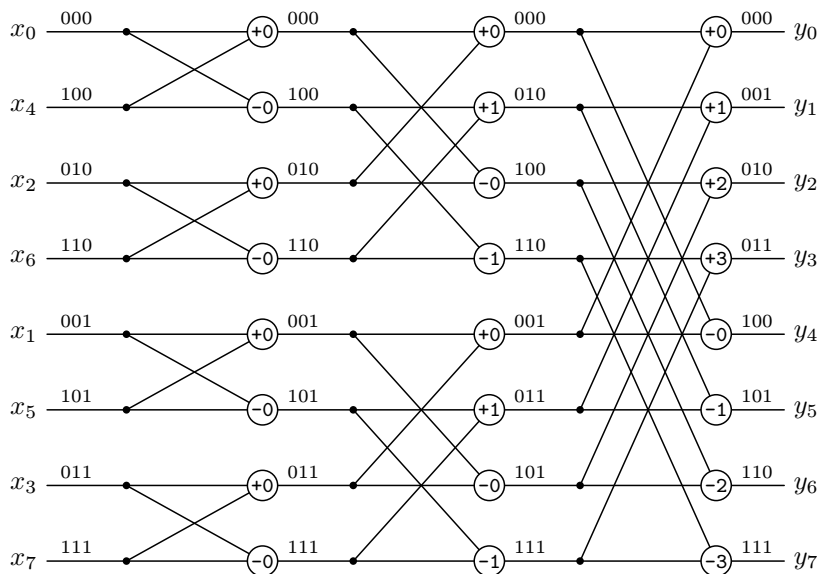
Ověřte, že i tato transformace je bijekce, a odvoďte algoritmus na její efektivní výpočet pomocí jednorozměrné FFT. Fyzikální interpretace je podobná: Fourierův obraz popisuje rozklad matice na „prostorové frekvence“. Také lze odvodit dvojměrnou cosinovou transformaci, na níž je založený například kompresní algoritmus JPEG. [N21]

## 17.5\* Další varianty FFT

### FFT jako hradlová síť

Zkusme průběh algoritmu FFT znázornit graficky. Na levé straně obrázku 17.4 se nachází vstupní vektor  $x_0, \dots, x_{n-1}$  (v nějakém pořadí), na pravé straně pak výstupní vektor  $y_0, \dots, y_{n-1}$ . Sledujme chod algoritmu pozpátku: Výstup spočítáme z výsledků „polovičních“ transformací vektorů  $x_0, x_2, \dots, x_{n-2}$  a  $x_1, x_3, \dots, x_{n-1}$ . Kroužky přitom odpovídají výpočtu lineární kombinace  $a + \omega^k b$ , kde  $a, b$  jsou vstupy kroužku ( $a$  přichází rovně,  $b$  šikmo) a  $k$  číslo uvnitř kroužku. Každá z polovičních transformací se počítá analogicky z výsledků transformace velikosti  $n/4$  atd. Celkově výpočet probíhá v  $\log_2 n$  vrstvách po  $\Theta(n)$  operacích.

Čísla nad čarami prozrazují, jak jsme došli k permutaci vstupních hodnot nalevo. Ke každému podproblému jsme napsali ve dvojkové soustavě indexy prvků vstupu, ze kterých podproblém počítáme. V posledním sloupci je to celý vstup. V předposledním nejprve indexy končící 0, pak ty končící 1. O sloupec vlevo jsme každou podrozdělili podle předposlední číslice atd. Až v prvním sloupci jsou čísla uspořádaná podle dvojkových zápisů čtených pozpátku. (Rozmyslete si, proč tomu tak je.)



Obrázek 17.4: Průběh FFT pro vstup velikosti 8

Na obrázek se také můžeme dívat jako na schéma hradlové sítě pro výpočet DFT. Kroužky jsou přitom „hradla“ pracující s komplexními čísly. Všechny operace v jedné vrstvě jsou na sobě nezávislé, takže je síť počítá paralelně. Síť tedy pracuje v čase  $\Theta(\log n)$  a prostoru  $\Theta(n \log n)$ .

### Nerekurzivní FFT

Obvod z obrázku 17.4 můžeme vyhodnocovat po hladinách zleva doprava. Tím získáme elegantní nerekurzivní algoritmus pro výpočet FFT. Pracuje v čase  $\Theta(n \log n)$  a prostoru  $\Theta(n)$ :

**Algoritmus FFT2** (rychlá Fourierova transformace nerekurzivně)

*Vstup:* Komplexní čísla  $x_0, \dots, x_{n-1}$ , primitivní  $n$ -tá odmocnina z jedné  $\omega$

1. Předpočítáme tabulku hodnot  $\omega^0, \omega^1, \dots, \omega^{n-1}$ .
2. Pro  $k = 0, \dots, n-1$  položíme  $y_k \leftarrow x_{r(k)}$ , kde  $r$  je funkce bitového zrcadlení.
3.  $b \leftarrow 1$   $\triangleleft$  velikost bloku
4. Dokud  $b < n$ , opakujeme:
5.     Pro  $j = 0, \dots, n-1$  s krokem  $2b$  opakujeme:  $\triangleleft$  začátek bloku
6.         Pro  $k = 0, \dots, b-1$  opakujeme:  $\triangleleft$  pozice v bloku
7.              $\alpha \leftarrow \omega^{(nk/2b) \bmod n}$
8.              $(y_{j+k}, y_{j+k+b}) \leftarrow (y_{j+k} + \alpha \cdot y_{j+k+b}, y_{j+k} - \alpha \cdot y_{j+k+b})$
9.      $b \leftarrow 2b$

*Výstup:*  $y_0, \dots, y_{n-1}$

### FFT v konečných tělesech

Nakonec dodejme, že Fourierovu transformaci lze zavést nejen nad tělesem komplexních čísel, ale i v některých konečných tělesech, pokud zaručíme existenci primitivní  $n$ -té odmocniny z jedničky. Například v tělese  $\mathbb{Z}_p$  pro prvočíslo  $p$  tvaru  $2^k + 1$  platí  $2^k = -1$ . Proto  $2^{2k} = 1$  a  $2^0, 2^1, \dots, 2^{2k-1}$  jsou navzájem různé. Číslo 2 je tedy primitivní  $2k$ -tá odmocnina z jedné. To se nám ovšem nehodí pro algoritmus FFT, neboť  $2k$  bude málokdy mocnina dvojky.

Zachrání nás ovšem algebraická věta, která říká, že multiplikativní grupa<sup>(2)</sup> libovolného konečného tělesa  $\mathbb{Z}_p$  je cyklická, tedy že všech  $p-1$  nenulových prvků tělesa lze zapsat jako mocniny nějakého čísla  $g$  (generátoru grupy). Jelikož mezi čísla  $g^0, g^1, g^2, \dots, g^{p-2}$  se každý nenulový prvek tělesa vyskytne právě jednou, je  $g$  primitivní  $(p-1)$ -ní odmocninou z jedničky.

<sup>(2)</sup> To je množina všech nenulových prvků tělesa s operací násobení.

V praxi se hodí například tyto hodnoty:

- $p = 2^{16} + 1 = 65\,537$ ,  $g = 3$ , takže funguje  $\omega = 3$  pro  $n = 2^{16}$  (analogicky  $\omega = 3^2$  pro  $n = 2^{15}$  atd.),
- $p = 15 \cdot 2^{27} + 1 = 2\,013\,265\,921$ ,  $g = 31$ , takže pro  $n = 2^{27}$  dostaneme  $\omega = g^{15} \bmod p = 440\,564\,289$ .
- $p = 3 \cdot 2^{30} + 1 = 3\,221\,225\,473$ ,  $g = 5$ , takže pro  $n = 2^{30}$  vyjde  $\omega = g^3 \bmod p = 125$ .

Bližší průzkum našich úvah o FFT dokonce odhalí, že není ani potřeba tělesa. Postačí libovolný komutativní okruh, ve kterém existuje příslušná primitivní odmocnina z jedničky, její multiplikativní inverze (ta ovšem existuje vždy, protože  $\omega^{-1} = \omega^{n-1}$ ) a multiplikativní inverze čísla  $n$ . To nám poskytuje ještě daleko více volnosti než tělesa, ale není snadné takové okruhy hledat.

Výhodou těchto podob Fourierovy transformace je, že na rozdíl od té klasické komplexní nejsou zatíženy zaokrouhlovacími chybami (komplexní odmocniny z jedničky mají obě složky iracionální). To se hodí například v algoritmech na násobení velkých čísel – viz cvičení 2.

### Cvičení

1. Navrhněte, jak počítat bitové zrcadlení v algoritmu FFT2.
- 2.\* *Násobení čísel:* Pomocí FFT lze rychle násobit čísla. Každé  $n$ -bitové číslo  $x$  můžeme rozložit na  $k$ -bitové bloky  $x_0, \dots, x_{m-1}$  (kde  $m = \lceil n/k \rceil$ ). To je totéž, jako kdybychom ho zapsali v soustavě o základu  $B = 2^k$ :  $x = \sum_j x_j B^j$ . Pokud k číslu přiřadíme polynom  $X(t) = \sum_j x_j t^j$ , bude  $X(B) = x$ .

To nám umožňuje převést násobení čísel na násobení polynomů: Chceme-li vynásobit čísla  $x$  a  $y$ , sestrojíme polynomy  $X$  a  $Y$ , pomocí FFT vypočteme jejich součin  $Z$  a pak do něj dosadíme  $B$ . Dostaneme  $Z(B) = X(B) \cdot Y(B) = xy$ .

Na RAMu přitom můžeme zvolit  $k \in \Theta(\log n)$ , takže s čísly polynomiálně velkými vzhledem k  $B$  zvládneme počítat v konstantním čase. Proto FFT ve vhodném konečném tělese poběží v čase  $\mathcal{O}(m \log m) = \mathcal{O}(n/\log n \cdot \log(n/\log n)) \subseteq \mathcal{O}(n/\log n \cdot \log n) = \mathcal{O}(n)$ . Zbývá domyslet, jak vyhodnotit  $Z(B)$ . Spočítejte, jak velké jsou koeficienty polynomu  $Z$ , a ukažte, že při vyhodnocování od nejnižších řádů jsou přenosy dostatečně malé na to, abychom výpočet  $Z(B)$  stihli v čase  $\Theta(n)$ .

Tím jsme získali algoritmus na násobení  $n$ -bitových čísel v čase  $\Theta(n)$ .

## **18 Pokročilé haldy**

\_\_\_\_\_

## 18 Pokročilé haldy

Při analýze Dijkstrova algoritmu v oddílu 6.2 jsme zatoužili po haldě, která by některé operace uměla rychleji než obyčejná binární halda. V této kapitole postupně odvodíme tři datové struktury: binomiální haldu, línou binomiální haldu a Fibonacciho haldu. Poslední z nich bude mít vytoužené vlastnosti.

### 18.1 Binomiální haldy

Základní funkce binomiální haldy jsou podobné binární haldě, nicméně jich dosahuje jinými metodami. Navíc podporuje operaci MERGE, která umí rychle sloučit dvě binomiální haldy do jedné.

Shrňme na začátek podporované operace spolu s jejich worst-case časovými složitostmi (tedy složitostmi v nejhorším případě). Číslo  $n$  udává počet prvků v haldě a haldu zde chápeme jako minimovou.

| <i>operace</i> | <i>složitost</i> | <i>činnost</i>                   |
|----------------|------------------|----------------------------------|
| INSERT         | $\Theta(\log n)$ | vloží nový prvek                 |
| MIN            | $\Theta(1)$      | vrátí minimum množiny            |
| EXTRACTMIN     | $\Theta(\log n)$ | vrátí a odstraní minimum množiny |
| MERGE          | $\Theta(\log n)$ | sloučí dvě haldy do jedné        |
| BUILD          | $\Theta(n)$      | postaví z $n$ prvků haldu        |
| DECREASE       | $\Theta(\log n)$ | sníží hodnotu klíče prvku        |
| INCREASE       | $\Theta(\log n)$ | zvýší hodnotu klíče prvku        |
| DELETE         | $\Theta(\log n)$ | smaže prvek                      |

Nadále platí, že podle klíče neumíme vyhledávat, takže operace DECREASE, INCREASE a DELETE musí dostat ukazatel na prvek v haldě, nikoliv jeho klíč. Ukazatel se dozvíme při vložení do haldy INSERTem.

Nyní binomiální haldu definujeme. Na rozdíl od binární haldy nebude mít tvar stromu, nýbrž souboru více tzv. binomiálních stromů.

#### Binomiální stromy

**Definice:** Řekneme, že zakořeněný strom  $T$  je *binomiálním stromem řádu  $k$* , pokud splňuje následující pravidla:

1. Pokud je řád  $k$  roven nule, pak  $T$  obsahuje pouze kořen.
2. Pokud je řád  $k$  nenulový, pak  $T$  má kořen s právě  $k$  syny. Tito synové jsou kořeny podstromů, které jsou po řadě binomiálními stromy řádů  $0, \dots, k-1$ .